JAVA PROGRAMMING PRACTICAL RECORD

Week 6 — Longest Substring Without Repeating Characters

Experiment No.	Week 6 / _____
Date	____ / ____ / _____
Title	Find the longest substring without repeating characters

1. Aim / Objective

To write and execute a Java program to find the **longest substring without repeating characters** in a given string. The program uses a `HashSet` and a sliding-window technique to keep track of unique characters and determine the maximum substring length.

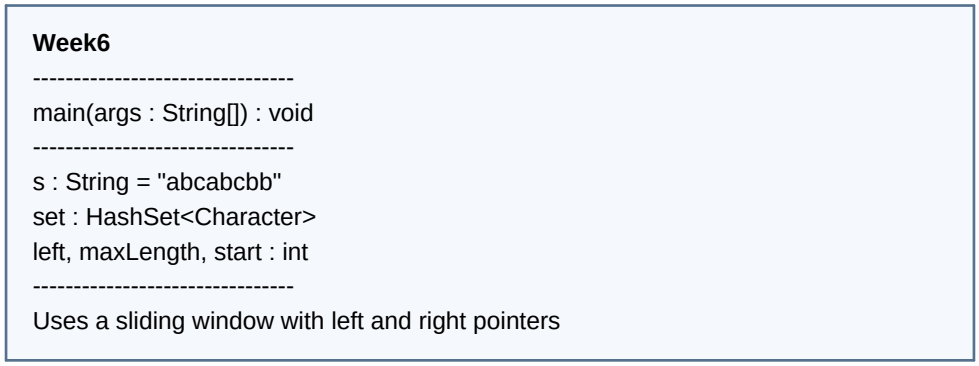
2. System Requirements

Hardware	Standard PC or laptop, Core i3/i5 or equivalent processor, and at least 4 GB RAM.
Software	Windows, Linux, or macOS; JDK 17 or JDK 21 recommended; any text editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Start the program and initialize the input string as `"abcabcbb"`.
2. Create a `HashSet<Character>` to store the unique characters in the current window.
3. Initialize `left`, `maxLength`, and `start` to zero.
4. Move the `right` pointer from the beginning to the end of the string.
5. For each character, while it already exists in the set, remove the character at the left pointer and move `left` one position forward.
6. Add the current character to the set. The window from `left` to `right` now contains no repeated characters.
7. If the current window is longer than `maxLength`, update the maximum length and save its starting index.
8. Extract the longest substring using `substring(start, start + maxLength)`.
9. Print the maximum length and the substring, then stop.

4. UML / Logic Diagram



Logic flow: initialize → scan characters with right pointer → remove duplicates by moving left pointer → update best window → extract and print result. This program does not use class inheritance or interfaces.

5. Source Code

Save the file as **Week6.java** inside a folder named **record** if you keep the `package record;` declaration. The source code below follows your submitted logic.

```
package record;

import java.util.*;

public class Week6 {
    public static void main(String args[]) {
        String s = "abcabcbb";

        HashSet<Character> set = new HashSet<>();
        int left = 0;
        int maxLength = 0;
        int start = 0;

        for (int right = 0; right < s.length(); right++) {
            char ch = s.charAt(right);

            while (set.contains(ch)) {
                set.remove(s.charAt(left));
                left++;
            }

            set.add(ch);

            if (right - left + 1 > maxLength) {
                maxLength = right - left + 1;
                start = left;
            }
        }

        String result = s.substring(start, start + maxLength);

        System.out.println("Length: " + maxLength);
        System.out.println("Substring: " + result);
    }
}
```

6. Compilation and Execution Commands

Because the code declares `package record;`, run these commands from the parent directory that contains the **record** folder:

```
javac record/Week6.java
java record.Week6
```

If your teacher wants to run `java Week6` directly, remove the package declaration and compile/run the file from the same directory using `javac Week6.java` and `java Week6`.

7. Expected Output Screen

```
Length: 3
Substring: abc
```

For submission, you can replace this text output box with a screenshot of the actual terminal after running the program.

8. Result

The program successfully finds the longest substring without repeating characters in the input string `"abcabcbb"`. The maximum length is **3**, and the substring returned is **"abc"**. The sliding-window approach processes the string efficiently in linear time on average.